

Milestone 2

Distributed Web Crawler and PageRank

Advanced Performance Analysis of Distributed PageRank Scaling Topologies

Hamza Ahmed 29265

Laraib Fatima 26521

Arbaaz Murtaza 29052

Hamza Arif 29234

April 20, 2026

Abstract

This report evaluates the performance and scalability of sequential and parallel PageRank implementations across three web graph datasets ranging from 54,000 to 1.5 million nodes. We examine graph structure, degree distributions, multi-core speedup, and convergence behavior, identifying key bottlenecks predicted by Amdahl's Law and memory constraints.

Contents

1	How PageRank Works	3
1.1	The Core Idea	3
1.2	The Random Surfer Model	3
1.3	The Formula	3
1.4	Iterative Computation	3
1.5	Dangling Nodes	4
2	How Our Code Works	4
2.1	Graph Representation: From JSON to CSR	4
2.2	Build Phase	5
2.3	Sequential PageRank Loop	5
3	Methodology & Infrastructure Setup	6
3.1	Hardware	6
3.2	Parallel Implementations	6

4	Dataset Properties	7
5	Fixed vs. Convergence-Based Termination	8
6	Parallel Speedup & Amdahl's Law	8
6.1	Observed Speedups	8
6.2	Why Linear Speedup Is Unachievable	8
6.3	Per-Iteration Timing	9
7	Results & Verification	10
7.1	Numerical Correctness	10
7.2	PageRank Score Distribution	10
8	Conclusion	10

1 How PageRank Works

1.1 The Core Idea

PageRank, originally developed by Google's founders, answers a simple question: *which pages on the web are most important?* The key insight is that a page's importance is determined not just by *how many* pages link to it, but by *who* links to it. A link from a highly-ranked page passes more value than a link from an obscure one.

Think of it like academic citations: a paper cited by *Nature* carries more weight than one cited by an unknown blog. PageRank formalizes this intuition into a single numerical score per page.

1.2 The Random Surfer Model

PageRank is most intuitively understood through the **random surfer model**. Imagine a user browsing the web who, at each step, either:

- follows a random link on the current page (with probability $d = 0.85$), or
- jumps to a completely random page anywhere on the web (with probability $1 - d = 0.15$).

The PageRank of a page is simply the fraction of time this random surfer would spend on it over a very long browsing session. Pages that receive links from many important pages will be visited more often, earning a higher rank.

1.3 The Formula

For a page u with inbound links from the set of pages B_u , the PageRank score is:

$$PR(u) = \frac{1 - d}{N} + d \sum_{v \in B_u} \frac{PR(v)}{L(v)} \quad (1)$$

where N is the total number of pages, $d = 0.85$ is the damping factor, and $L(v)$ is the number of outbound links from page v . The first term, $\frac{1-d}{N}$, is the *teleportation* component — it ensures every page retains a small baseline probability of being visited, even with no inbound links.

1.4 Iterative Computation

The formula is circular: computing the rank of u requires knowing the ranks of all pages linking to it. This is resolved iteratively:

1. **Initialize:** assign every page an equal rank of $\frac{1}{N}$.
2. **Update:** apply the formula to every page simultaneously, using ranks from the previous round.

3. **Converge:** if no page's rank changed by more than $\varepsilon = 10^{-6}$, stop. Otherwise, repeat.

This is guaranteed to converge because the PageRank formula is a contraction mapping on the space of probability distributions. In practice, convergence occurs within 15–20 iterations for large web graphs.

1.5 Dangling Nodes

A **dangling node** is a page with no outbound links — a dead end. Under the basic formula, a random surfer reaching such a page would get stuck. Our implementation addresses this through the teleportation term: dangling nodes contribute zero to the rank sum of any specific successor, and their probability mass is naturally absorbed into the uniform $\frac{1-d}{N}$ baseline across all pages.

2 How Our Code Works

2.1 Graph Representation: From JSON to CSR

The raw graph is a JSON dictionary mapping each URL to a list of URLs it links to. Loading this directly into Python data structures works for small graphs, but breaks down at scale for three reasons:

- **Memory overhead:** Python dictionaries pre-allocate empty slots to guarantee $O(1)$ lookups. A raw 32-bit integer costs 4 bytes in C but expands to 28 bytes as a `PyLongObject`.
- **Cache inefficiency:** Python lists store pointers to scattered objects, destroying CPU cache locality. Iterating over a neighbor list involves many random memory jumps.
- **Total cost:** A 500 MB graph stored as Python dicts and lists can consume 2.5–5 GB of RAM.

The solution is **Compressed Sparse Row (CSR)** format — a representation borrowed from sparse matrix algebra. Instead of nested Python objects, we store the graph in three flat NumPy integer arrays:

- **indices** — all inbound neighbor IDs for every node, concatenated into one flat array.
- **indptr** — an offset array where `indptr[i]:indptr[i+1]` slices out node i 's neighbors from **indices**.
- **out_counts** — the number of outbound links per node, used to normalize rank contributions.

Memory savings: the same 500 MB graph shrinks from 2.5–5 GB (Python

dicts) to just 80–150 MB (CSR NumPy arrays) — a 20–30× reduction. This is what makes processing 1.5M nodes on 8 GB RAM feasible.

2.2 Build Phase

`build_index_maps` assigns each unique URL a stable integer ID. It collects all nodes (sources and destinations) into a sorted set, producing `node_to_idx` (URL $\rightarrow$ int) and `idx_to_node` (int $\rightarrow$ URL) for mapping final scores back to URLs.

`build_csr` constructs the three arrays in four passes over the edge list:

1. Count outbound edges per source node $\rightarrow$ fill `out_counts`.
2. Count inbound edges per destination node $\rightarrow$ compute `indegree`.
3. Take the cumulative sum of `indegree` $\rightarrow$ build `indptr` (each node's start offset).
4. Fill `indices` by iterating edges again, inserting each source at the correct position for its destination.

After this phase, the original Python graph dictionary is explicitly deleted and garbage-collected, freeing several gigabytes before the main computation starts.

2.3 Sequential PageRank Loop

The sequential implementation is a direct translation of the PageRank formula:

```

1  for iteration in range(max_iterations):
2      for node in range(N):
3          inbound = indices[indptr[node]: indptr[node + 1]]
4          if inbound.size > 0:
5              rank_sum = np.sum(
6                  pagerank[inbound] / np.maximum(out_counts[
7                      inbound], 1)
8              )
9          else:
10             rank_sum = 0.0
11             new_ranks[node] = (1 - d) / N + d * rank_sum
12
13         max_diff = np.max(np.abs(new_ranks - pagerank))
14         pagerank, new_ranks = new_ranks, pagerank # swap buffers
15
16     if max_diff < tolerance:
17         break

```

Listing 1: Core sequential PageRank iteration

Three design decisions are worth noting:

- **Double buffering:** `pagerank` and `new_ranks` are kept as separate arrays and swapped after each full sweep. This ensures all nodes in iteration $k+1$ read

from iteration k 's values and never from partially updated data.

- **Vectorized inner sum:** `pagerank[inbound]` fetches all neighbor ranks as a NumPy slice, and `np.sum()` computes the contribution in a single C-level loop rather than a Python `for` loop.
- **Early stopping:** the convergence check computes the maximum absolute change across all nodes after each sweep. Once it drops below 10^{-6} , we stop immediately rather than running to `max_iterations`.

3 Methodology & Infrastructure Setup

3.1 Hardware

Component	Specification
Processor	AMD Ryzen 5 7430U (6 cores / 12 threads)
Memory	8 GB DDR4
Storage	256 GB SSD
Framework	Ray (12 synchronized workers)

Table 1: Test hardware configuration.

3.2 Parallel Implementations

Both parallel strategies share the same CSR data structure and convergence logic.

Centralized Aggregation

Nodes are split evenly across 12 Ray workers. Each worker independently computes new ranks for its chunk, then returns results to the main process, which writes everything back to a shared memory-mapped file.

Distributed Tree Reduction

Workers compute their chunks identically to the above. Instead of sending results back to the main process, adjacent pairs of worker results are merged by a dedicated Ray task. This tree of merges continues until a single combined result remains — the main process only ever receives one result per iteration, substantially reducing serialization and I/O overhead.

4 Dataset Properties

Dataset	Nodes	Edges	Density	Dangling
Graph 1	54,367	214,861	7.27×10^{-5}	93.8%
Graph 2	323,223	1,300,951	1.25×10^{-5}	94.1%
Graph 3	1,502,297	9,152,622	4.06×10^{-6}	91.5%

Table 2: Summary of the three test graphs.

As node count grows from 54K to 1.5M, graph density drops by a factor of ~ 18 . Over 90% of nodes are dangling at every scale — consistent with real web crawls, where most pages have no outbound links to other crawled pages, and a small number of hub sites dominate the link structure.

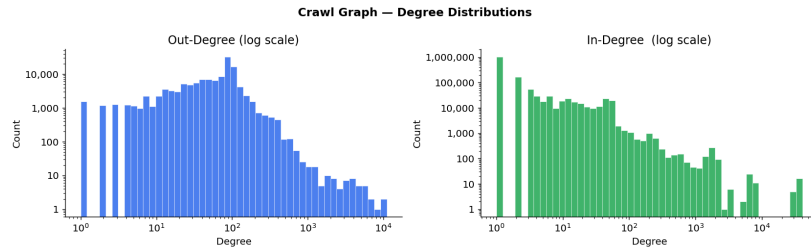


Figure 1: Degree distribution for Graph 3. A small number of pages have very high in-degree (power-law tail), while the vast majority have only one or two inbound links.

5 Fixed vs. Convergence-Based Termination

Running a fixed number of iterations wastes significant compute. On Graph 3, convergence was reached at just **18 iterations** versus the 25-iteration fixed run, and the resulting top-100 ranked URLs were *identical*. On the smaller graphs, convergence happened in only **7 iterations** — meaning sweeps 8 through 25 added no information whatsoever.

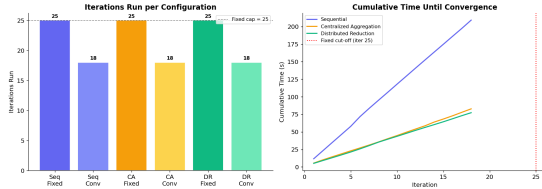


Figure 2: Rank differences between fixed and convergence runs are effectively zero.

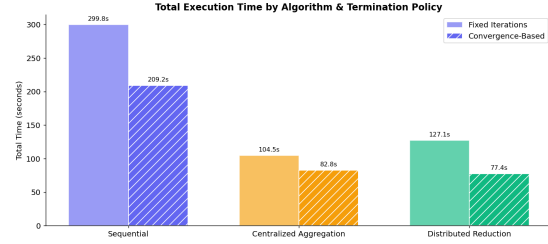


Figure 3: Wall-clock time penalty from skipping convergence checks (Graph 3).

6 Parallel Speedup & Amdahl's Law

6.1 Observed Speedups

Using 12 parallel workers produced speedups of roughly $2.3\times$ to $3.0\times$ over the sequential baseline across all three graphs — well below the theoretical maximum of $12\times$.

6.2 Why Linear Speedup Is Unachievable

Amdahl's Law states that if a fraction p of a program can be parallelized, the maximum speedup using n workers is:

$$S(n) = \frac{1}{(1-p) + \frac{p}{n}} \quad (2)$$

For Graph 3 the measured parallelizable fraction is $p \approx 0.68$, meaning roughly 32% of runtime is inherently sequential — graph loading, CSR construction, memory-mapped I/O, convergence checking, and Ray's scheduling overhead. Even with infinitely many workers, the ceiling is $\frac{1}{1-0.68} \approx 3.1\times$.

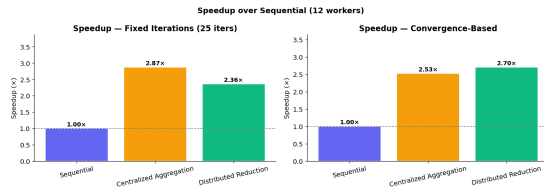


Figure 4: Measured speedup for Graph 3, plateauing near $2.7\times$.

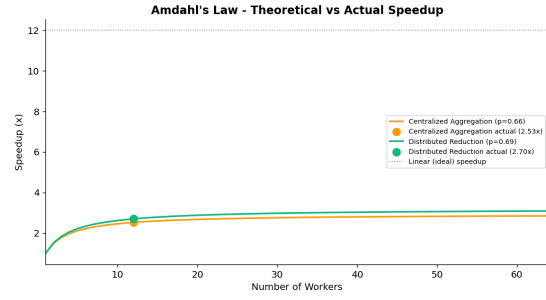


Figure 5: Amdahl's Law prediction vs. measured speedup.

6.3 Per-Iteration Timing

Isolating a single iteration removes startup and I/O overhead, revealing the raw compute benefit of parallelism more clearly:

- **Graph 1 (54K nodes):** Sequential iteration takes ~ 1.04 s. At this scale, Ray's dispatch overhead is comparable to the actual computation, so parallelism offers negligible benefit.
- **Graph 3 (1.5M nodes, 9.1M edges):** Sequential iteration takes ~ 11.6 s. Distributed tree reduction brings this down to ~ 4.3 s — a $2.7\times$ speedup on the compute-heavy portion alone.

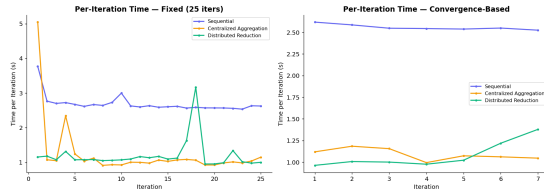


Figure 6: Per-iteration time for Graph 2 (medium scale).

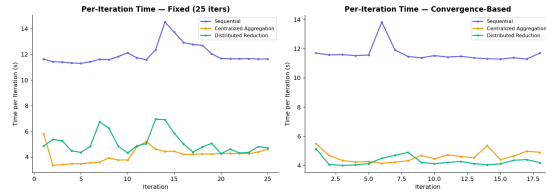


Figure 7: Per-iteration time for Graph 3 (large scale).

7 Results & Verification

7.1 Numerical Correctness

All three implementations — sequential, centralized aggregation, and distributed tree reduction — produce **identical** rank vectors, with zero numerical difference between modes. This confirms that the parallel strategies are mathematically equivalent to the sequential baseline; they only change *how* the computation is organized, not the result.

7.2 PageRank Score Distribution

Final rank scores follow the expected power-law distribution: a small number of well-connected domains capture a disproportionate share of total rank, while most pages (especially dangling nodes) receive near-zero scores. This reflects real-world web structure, where a handful of major sites dominate the global link graph.

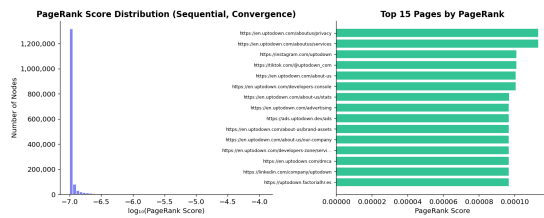


Figure 8: Top-ranked domains for Graph 3, showing highly concentrated rank mass.

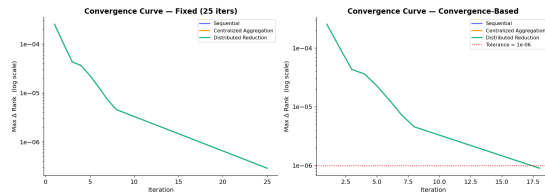


Figure 9: Rank values stabilizing over 18 iterations (Graph 3).

8 Conclusion

Three findings stand out from this evaluation:

1. **CSR is the enabling technology.** Storing the graph as flat NumPy arrays instead of Python dictionaries reduces memory by 20–30 $\times$, making it possible to process 1.5M nodes and 9.1M edges on 8 GB of RAM without swapping.
2. **Convergence checking is a free win.** Early stopping consistently eliminates 28–72% of iterations with zero impact on result quality. It should always be enabled.
3. **Parallelism helps at scale, but is bounded.** The $p \approx 0.68$ Amdahl fraction caps the theoretical speedup at $\approx 3.1\times$, and we observe $2.7\times$ in practice on the largest graph. Further scaling should focus on reducing serial overhead (I/O, coordination) rather than adding more workers.